

Frontend: Theory Assignment 2

1. What is CSS Box Model?

The **CSS Box Model** describes how every HTML element is represented as a rectangular box. It consists of four parts:

1. **Content** – The actual text, image, or other content.
2. **Padding** – Space between the content and border.
3. **Border** – A line surrounding the padding and content.
4. **Margin** – Space outside the border between elements.

Order:

Content → Padding → Border → Margin

2. What is Media Query? Explain with a sample code.

A **media query** is a CSS technique used to apply different styles to a webpage depending on the device's screen size, resolution, or other characteristics. It is commonly used to create **responsive websites**.

Example:

```
/* Normal style */
.container {
    width: 80%;
    margin: auto;
}

/* Style for screens 600px or smaller */
@media (max-width: 600px) {
    .container {
        width: 100%;
        font-size: 14px;
    }
}
```

Here, the styles inside `@media` are applied when the screen width is **600px or less**.

3. What are the features of Bootstrap?

The main features of **Bootstrap** are:

- Responsive design
- Mobile-first approach
- Grid system
- Predefined CSS classes
- Ready-made UI components
- JavaScript components
- Cross-browser compatibility
- Easy customization
- Consistent design
- Utility classes for spacing, colors, typography, etc.

4. Write an example of basic grid structure in Bootstrap.

A basic Bootstrap grid consists of **container, row, and columns**.

```
<div class="container">
  <div class="row">
    <div class="col-md-4">Column 1</div>
    <div class="col-md-4">Column 2</div>
    <div class="col-md-4">Column 3</div>
  </div>
</div>
```

The Bootstrap grid is based on **12 columns**. Here, each `col-md-4` occupies 4 out of 12 columns.

5. What is CSS Selectors?

CSS selectors are patterns used to select HTML elements to which CSS styles should be applied.

Common types of CSS selectors are:

- **Element selector:** p
- **Class selector:** .heading
- **ID selector:** #header
- **Universal selector:** *
- **Grouping selector:** h1, h2, p
- **Attribute selector:** input[type="text"]
- **Descendant selector:** div p

Example:

```
p {  
    color: blue;  
}  
  
.heading {  
    font-size: 20px;  
}  
  
#header {  
    background-color: black;  
}
```

6. What are CSS backgrounds? List the properties.

CSS backgrounds are used to define the background appearance of an HTML element.

Important CSS background properties are:

1. background-color
2. background-image
3. background-repeat
4. background-position
5. background-size
6. background-attachment
7. background-origin
8. background-clip

Example:

```
body {  
    background-color: lightblue;  
    background-image: url("image.jpg");  
    background-repeat: no-repeat;  
    background-position: center;  
    background-size: cover;  
}
```

7. How can we overlap elements in CSS?

Elements can be overlapped using **positioning** and the **z-index** property.

```
.box1 {  
    position: absolute;  
    top: 50px;  
    left: 50px;  
    width: 100px;  
    height: 100px;  
    background: red;  
}
```

```
.box2 {  
    position: absolute;  
    top: 80px;  
    left: 80px;  
    width: 100px;  
    height: 100px;  
    background: blue;  
    z-index: 2;  
}
```

Here, the two elements overlap because they use `position: absolute`. The `z-index` determines which element appears on top.

8. What are the different positioning properties in CSS?

CSS provides the following positioning properties:

1. **Static** – Default position of an element.
2. **Relative** – Positions an element relative to its normal position.
3. **Absolute** – Positions an element relative to its nearest positioned ancestor.
4. **Fixed** – Positions an element relative to the browser viewport.
5. **Sticky** – Acts like relative positioning until a specified scroll position is reached, then behaves like fixed positioning.

Example:

```
.static {  
    position: static;  
}  
  
.relative {  
    position: relative;  
}  
  
.absolute {  
    position: absolute;  
}  
  
.fixed {  
    position: fixed;  
}  
  
.sticky {  
    position: sticky;  
}
```

9. What is ECMAScript?

ECMAScript is a standardized scripting language specification that defines the rules and features of scripting languages such as **JavaScript**.

JavaScript follows the **ECMAScript standard**. ECMAScript specifies features such as variables, functions, objects, classes, modules, and syntax.

10. What is the difference between **var**, **let** and **const** in JavaScript? Explain Temporal Dead Zone.

Difference between var, let, and const

Feature	var	let	const
Scope	Function scope	Block scope	Block scope
Redeclaration	Allowed	Not allowed	Not allowed
Reassignment	Allowed	Allowed	Not allowed
Hoisting	Hoisted and initialized as undefined	Hoisted but not initialized	Hoisted but not initialized

Example:

```
var x = 10;
var x = 20;    // Allowed
```

```
let y = 10;
y = 20;        // Allowed
```

```
const z = 10;
z = 20;        // Error
```

Temporal Dead Zone (TDZ)

The **Temporal Dead Zone** is the period between entering a block and the point where a **let** or **const** variable is declared and initialized.

During this period, accessing the variable causes a **ReferenceError**.

```
console.log(x); // ReferenceError
let x = 10;
```